



On the Minimum Quartet Tree Cost problem for the quartet method of hierarchical clustering

Sergio Consoli

European Commission, Joint Research Centre (JRC)

Ispra (VA), Italy

[name.surname]@ec.europa.eu

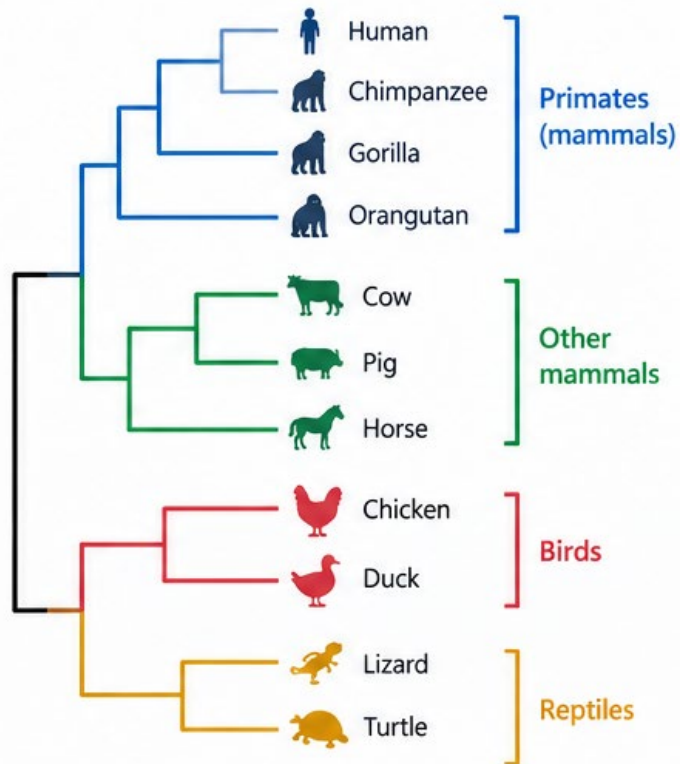
BENQODHI 2026 Workshop 2026, 10-11 Sept

Disclaimer: The views and opinions expressed in this presentation are those of the presenter and do not necessarily reflect the policies and opinions of the European Commission.

Joint
Research
Centre

Biology

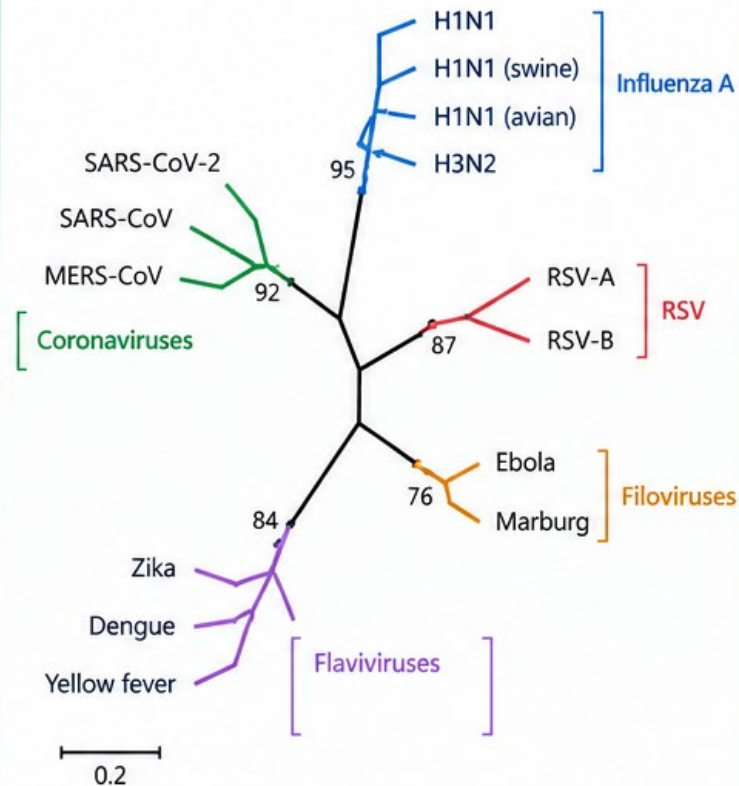
Phylogenetic tree of selected species



A phylogenetic tree shows the evolutionary relationships between species based on genetic or other biological data.

Genomics

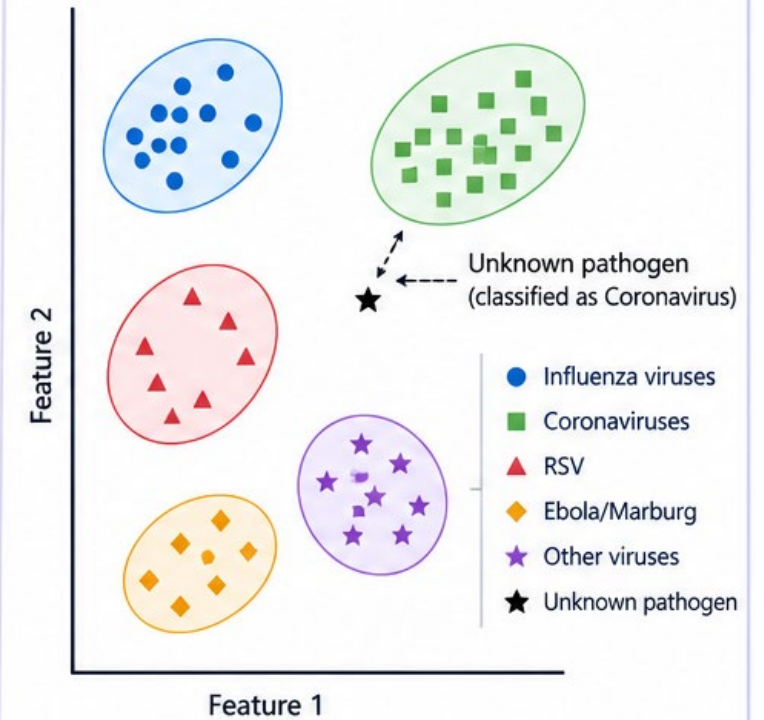
Phylogenetic tree of viral genomes



Genomic data can be used to reconstruct phylogenetic trees, revealing the evolutionary relationships between viruses (or other genomes).

Virology

Classifying unknown pathogens



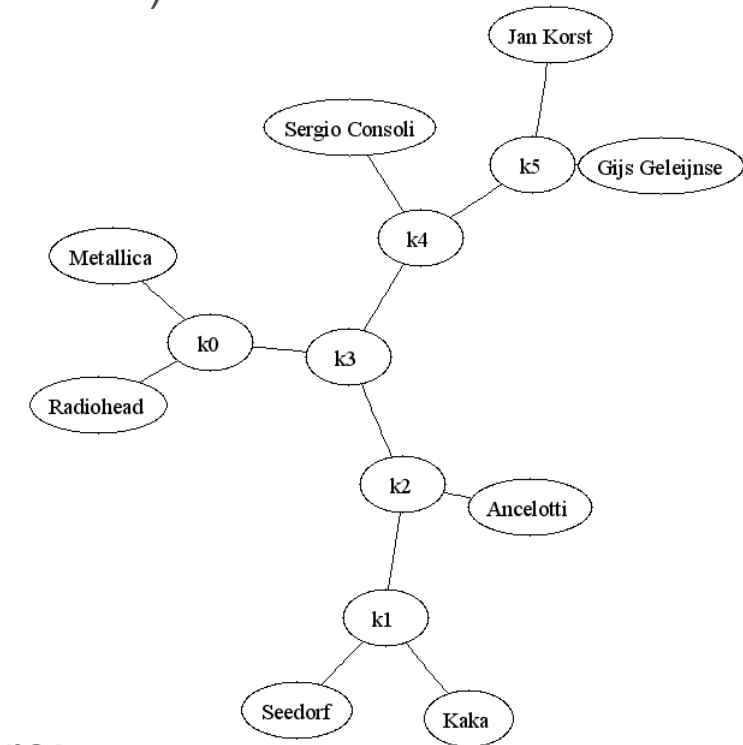
Distance-based methods can classify unknown pathogens by grouping them with known classes.

Quartet method

INPUT: given n elements, a *symmetric distance matrix* $n \times n$

OUTPUT: a *full unrooted binary tree* (or *trivalent tree*, or *boron tree*) with n leaves

	Kaka	Jan Korst	Ancelotti	Seedorf	Sergio	Radiohead	Gijs	Metallica
Kaka	0.00	1.00	0.30	0.20	0.80	1.00	0.90	1.00
Jan Korst	1.00	0.00	1.00	1.00	0.20	0.80	0.05	0.90
Ancelotti	0.30	1.00	0.00	0.40	0.60	1.00	0.80	0.95
Seedorf	0.20	1.00	0.40	0.00	0.89	0.99	0.90	1.00
Sergio	0.80	0.20	0.60	0.89	0.00	0.93	0.30	1.00
Radiohead	1.00	0.80	1.00	0.99	0.93	0.00	0.90	0.05
Gijs	0.90	0.05	0.80	0.90	0.30	0.90	0.00	0.55
Metallica	1.00	0.90	0.95	1.00	1.00	0.05	0.55	0.00

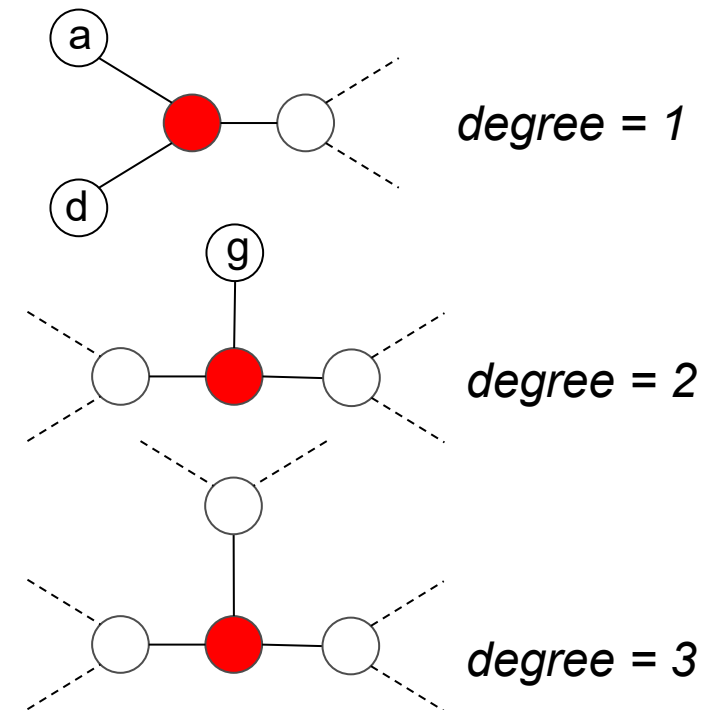


- Why full binary trees? Because they have a rich internal structure: many differentiated paths between the leaves
- **GOAL:** find the **full binary tree** with n leaves that much agrees with the **distance matrix** (according to a certain cost function)

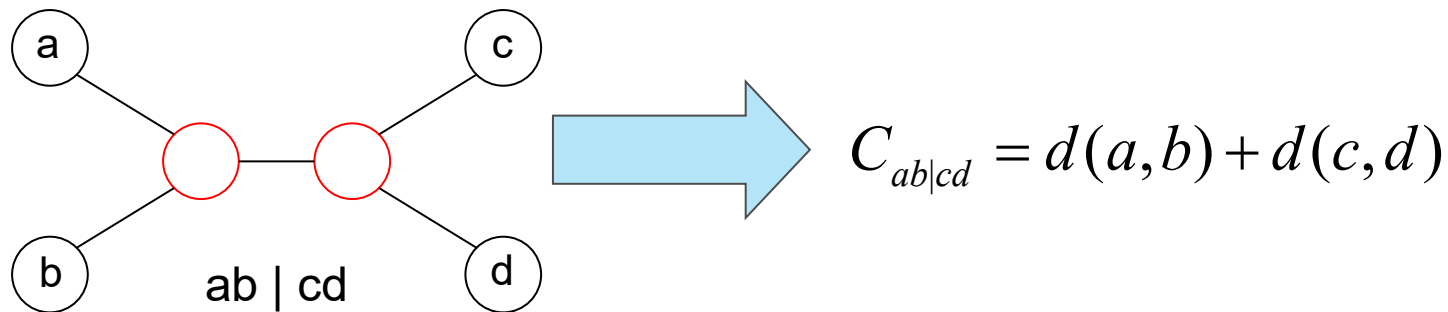
Definitions

Tree possible internal nodes:

1. *Terminal node*: connected with 2 leaves and 1 internal node
2. *Transition node*: connected with 1 leaf and 2 internal nodes
3. *Cross node*: connected with 3 internal nodes (no leaves)

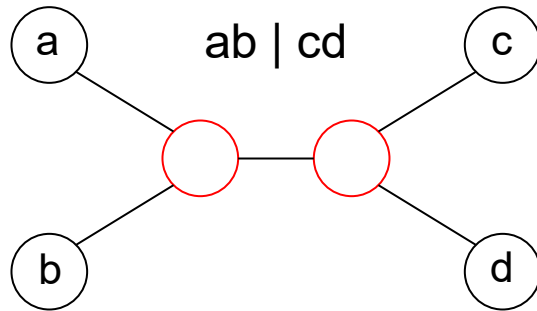


Cost associate to a *single quartet topology* (i.e. simplest full binary tree with $n=4$ leaves and $n-2=2$ internal nodes, e.g. $ab|cd$):

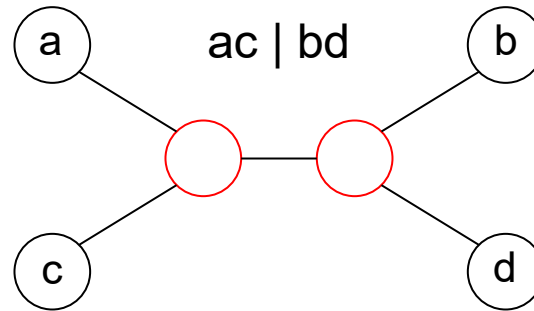


Mathematical paradigm

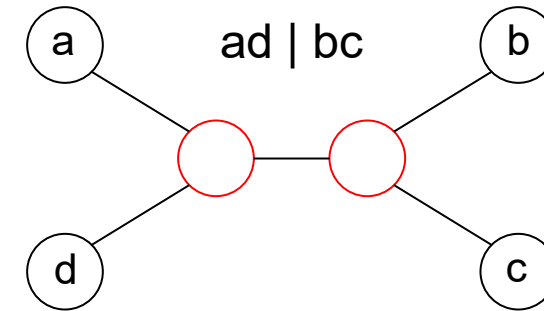
- Given n elements, consider all the set of 4 elements from our set of n elements
- There are $\binom{n}{4} = \frac{n!}{4!(n-4)!} = \frac{n(n-1)(n-2)(n-3)}{24}$ such sets
- For each set of 4 elements, there are exactly three possible combinations



$$C_{ab|cd} = d(a,b) + d(c,d)$$



$$C_{ac|bd} = d(a,c) + d(b,d)$$



$$C_{ad|bc} = d(a,d) + d(b,c)$$

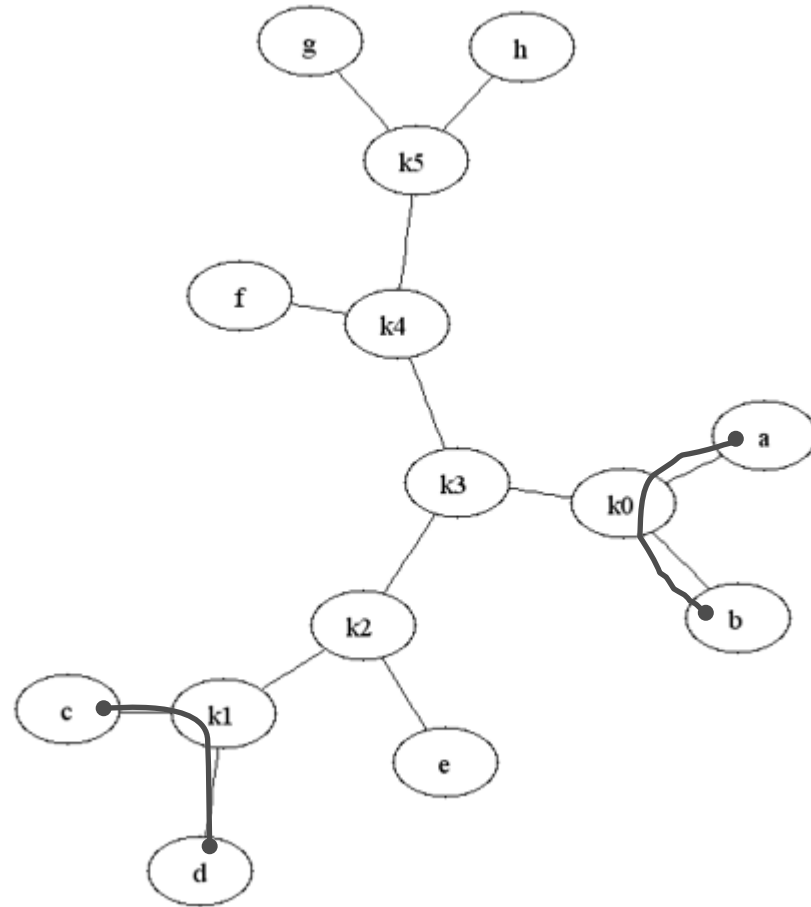
- Thus, given n elements, the total number of possible quartet topologies is: $3 \cdot \binom{n}{4}$

Consistency

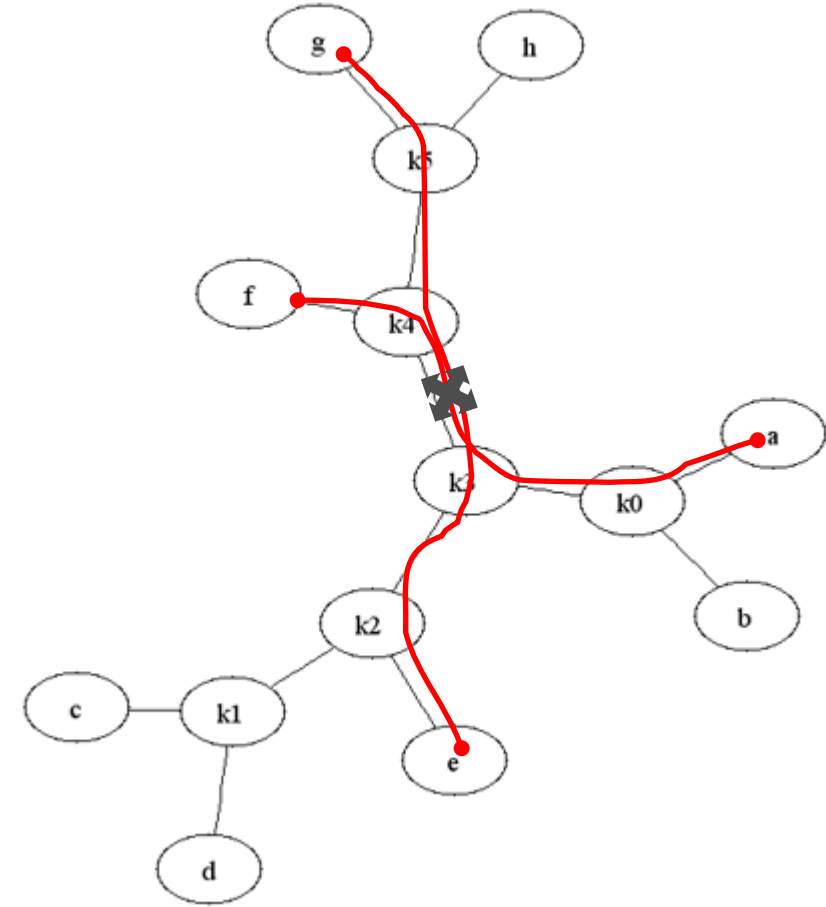
- Given n elements, consider the class T of full binary trees obtained placing the n elements as leaf nodes (i.e. $(n-2)$ internal nodes)
- Given a tree $t \in T$, we say that t is *consistent* to the quartet “ab|cd” if and only if the path from a to b does not cross the path from c to d
 - N.B. Precisely one of the three possible quartet topologies for any set of 4 leaves is consistent for a given tree $t \in T$
- Thus, for each $t \in T$, there exists precisely $\binom{n}{4} = \frac{n(n-1)(n-2)(n-3)}{24}$ consistent quartet topologies (one for each set of 4 elements) embedded in t

Consistency examples

- t is *consistent* to the quartet “ab|cd”



- t is NOT *consistent* to the quartet “af|ge”



Minimum Quartet Tree Cost problem

- **DEF.** Cost associated to $t \in T$: sum of the costs of its embedded quartet topologies

$$C_t = \sum_{\{abcd\}} C_{ab|cd} / ab | cd \text{ is consistent with } t$$

- **DEF. MINIMUM QUARTET TREE COST (MQTC) PROBLEM**

Given n elements and their pairwise distances, find the tree $t \in T$ with the minimum total cost C_t

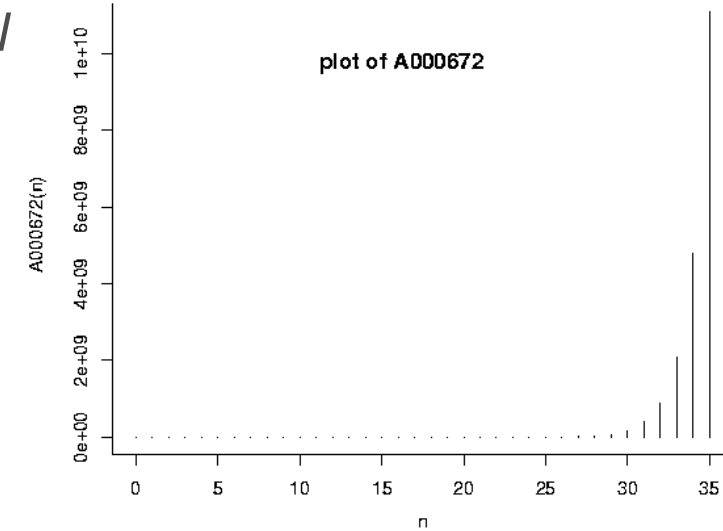
$$\min C_t = \min \sum_{\{abcd\}} C_{ab|cd} / ab | cd \text{ is consistent with } t$$

- NP-hard optimization problem (Cilibrasi and Vitany, 2005; 2007)

Computational complexity

1. Given a *distance matrix* with n elements, how many **different full binary trees** with n leaves ($n - 2$ internal nodes) is possible to generate?

- This sequence is referred in Sloane and Plouffe (1995)'s Encyclopedia of Integer Sequences to as A000672.
- Exponential growth (Sloane's A000672)



<i>n</i> leaves	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...	30
topologies	1	1	2	2	4	6	11	18	37	66	135	265	552	1132	2·10 ³	5·10 ³	11·10 ³	...	32·10⁶

2. Given a *distance matrix* with n elements, for each different *full binary tree*, considering all the possible **permutations** of its leaves (elements in the matrix), is it possible to find the best solution obtained by the considered binary tree?

- Factorial growth: $n!$

Normalization of the cost function

Calculate the worst (maximal) cost M :

$$M = \sum_{\{abcd\}} M_{abcd} / \text{for each combin. of four elements}$$

Calculate the best (minimal) cost m :

$$m = \sum_{\{abcd\}} m_{abcd} / \text{for each combin. of four elements}$$

M upper bound, m lower bound for C_t : $m \leq C_t \leq M \quad \forall t$

In most of cases, the best cost m cannot be really attained by any boron tree t , especially for a large number of elements $n \rightarrow$ **INCONSISTENCY**

Normalized tree score $S(t)$: $S(t) = \frac{M - C_t}{M - m} \in [0, 1] \quad \forall t$

MQTC approximate optimization routines

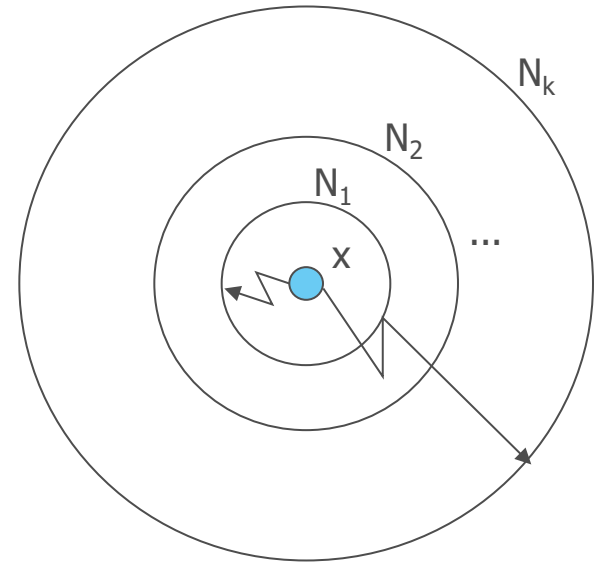
- Randomized Hill-Climbing, Greedy Randomized Adaptive Search Procedure, Genetic Programming, Particle Swarm Optimization, Simulated Annealing, Variable Neighborhood Search

The image displays three overlapping screenshots of academic publications. The top-left screenshot shows the abstract of 'Heuristic Approaches for the Quartet Method of Hierarchical Clustering' from IEEE Transactions on Knowledge and Data Engineering, Volume 22, Issue 10, published in 2009. The middle screenshot shows the abstract of 'A quartet method based on variable neighborhood search for biomedical literature extraction and clustering' from International Transactions in Operational Research, Volume 24, Issue 3, May 2017. The bottom-right screenshot shows the abstract of 'Improved metaheuristics for the quartet method of hierarchical clustering' from the Journal of Global Optimization, Volume 78, pages 241-270, published in 2020. Each screenshot includes the journal cover, title, authors, and abstract text.

- S. Consoli, J. Korst, S. Pauws, and G. Geleijnse, 2020. Improved metaheuristics for the quartet method of hierarchical clustering. *Journal of Global Optimization*, 78(2), 241-270
- S. Consoli and N. I. Stilianakis, 2017. A quartet method based on variable neighbourhood search for biomedical literature extraction and clustering. *International Transactions in Operational Research*, 24(3), 537-558
- Consoli S, Darby-Dowman K, Geleijnse G, Korst J, Pauws S (2010) Heuristic approaches for the quartet method of hierarchical clustering. *IEEE Transactions on Knowledge and Data Engineering* 22(10):1428– 1443

Variable Neighbourhood Search (VNS)

- VNS is based on dynamically changes neighbourhood structures during the search process (Hansen and Mladenović, 2001)
- Search for new solutions in increasingly distant neighbourhoods of the current solution, jumping only if a better solution is found
- DEF. $\mathbf{N}_k(\mathbf{x}) \rightarrow$ set of solutions in the k^{th} neighbourhood of $\mathbf{x}$
- *Shaking phase*
 - Apply k consecutive base moves
 - Base move: change the physical structure of the full binary tree
 - $k_{\max}$ = maximum size of the shaking phase →
 - maximum number of consecutive base moves
- *Local search phase*
 - First-improvement strategy
 - One-neighbourhood structure:
 - Considers internal nodes with Manhattan distance = 1
 - Performs a basic move among the two selected internal nodes

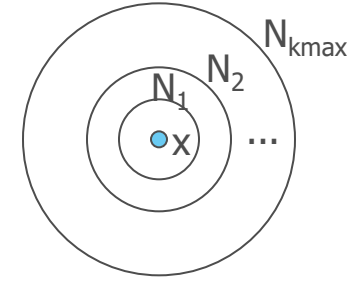


Variable Neighbourhood Search (VNS)

Initialisation

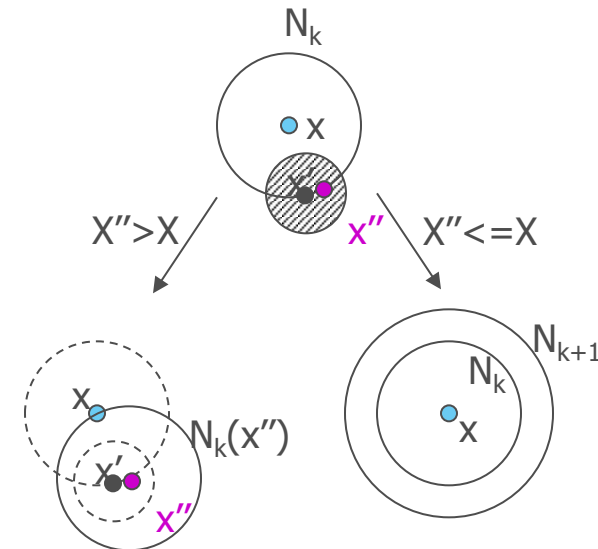
Select the set of neighbourhood structures N_k

Find an initial solution x



Repeat until stopping condition is met

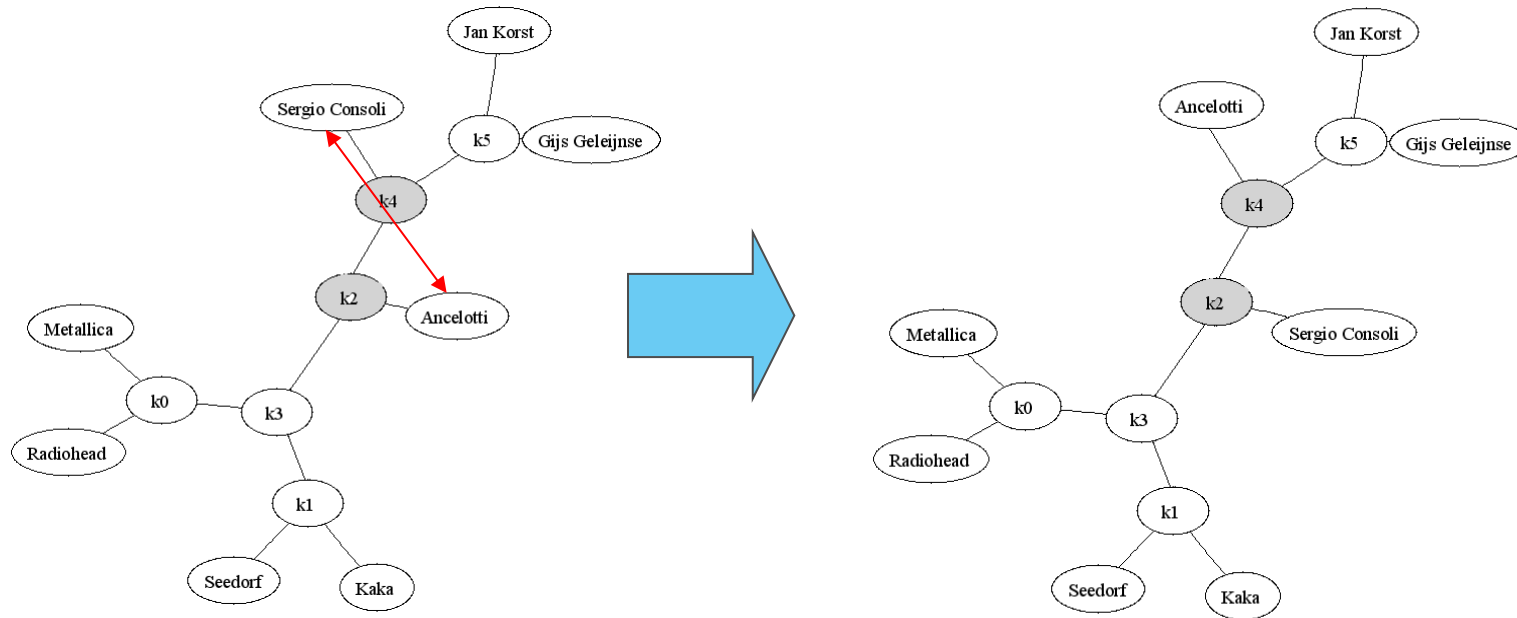
- Set $k=1$
- **Repeat** until $k=k_{max}$
 1. *Shaking*: Generate a random point x' in $N_k(x)$
 2. *Local Search*: x'' is the obtained optimum
 3. Move or not:
 - If x'' is better than x then $x \leftarrow x''$ and $k=1$
 - Otherwise $k=k+1$



Base moves

Two consecutive *transition nodes*

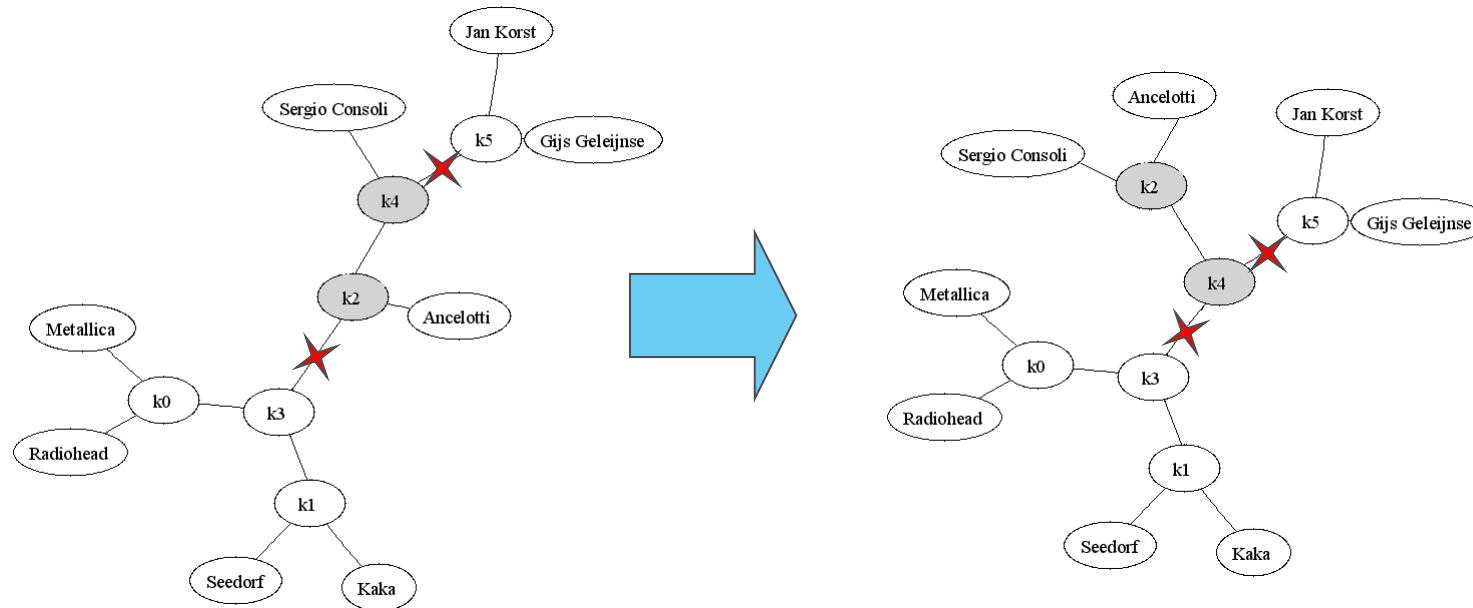
→ Exchange the leaves attached



Base moves

Two consecutive *transition nodes*

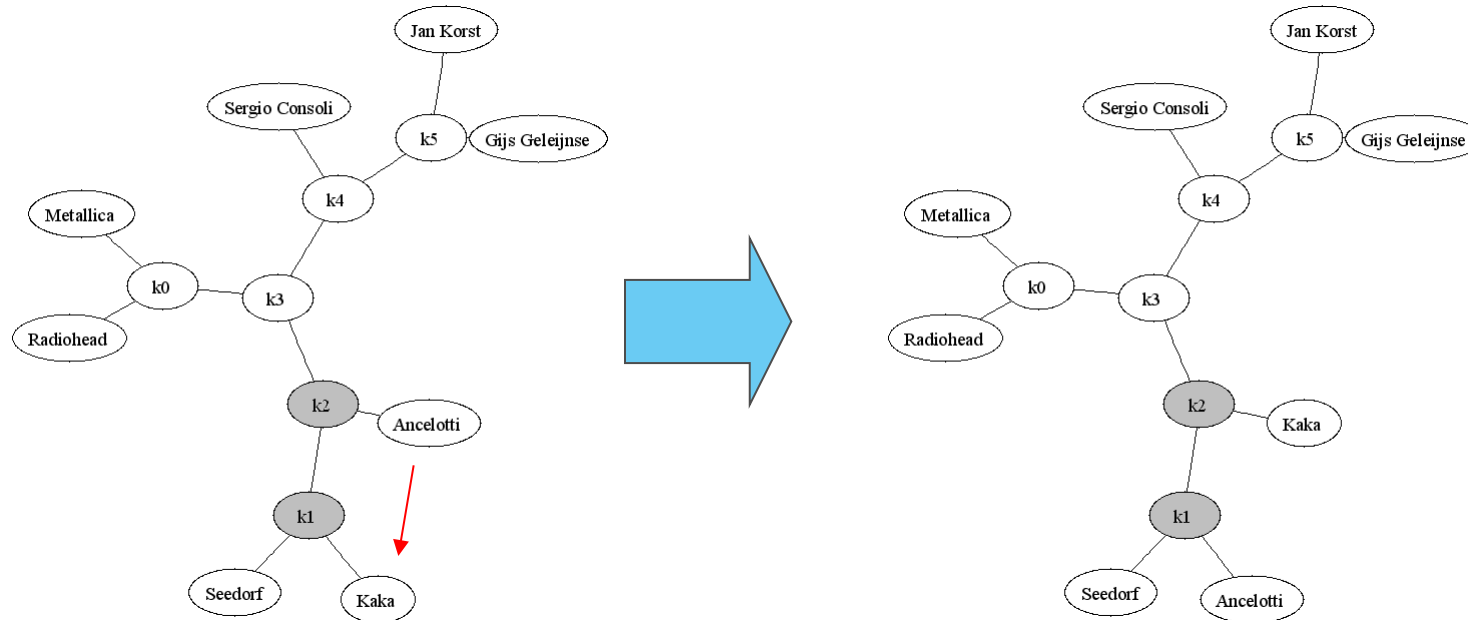
→ Transform in one *terminal node* and one *cross node*



Base moves

One *terminal node* and one *transition node*

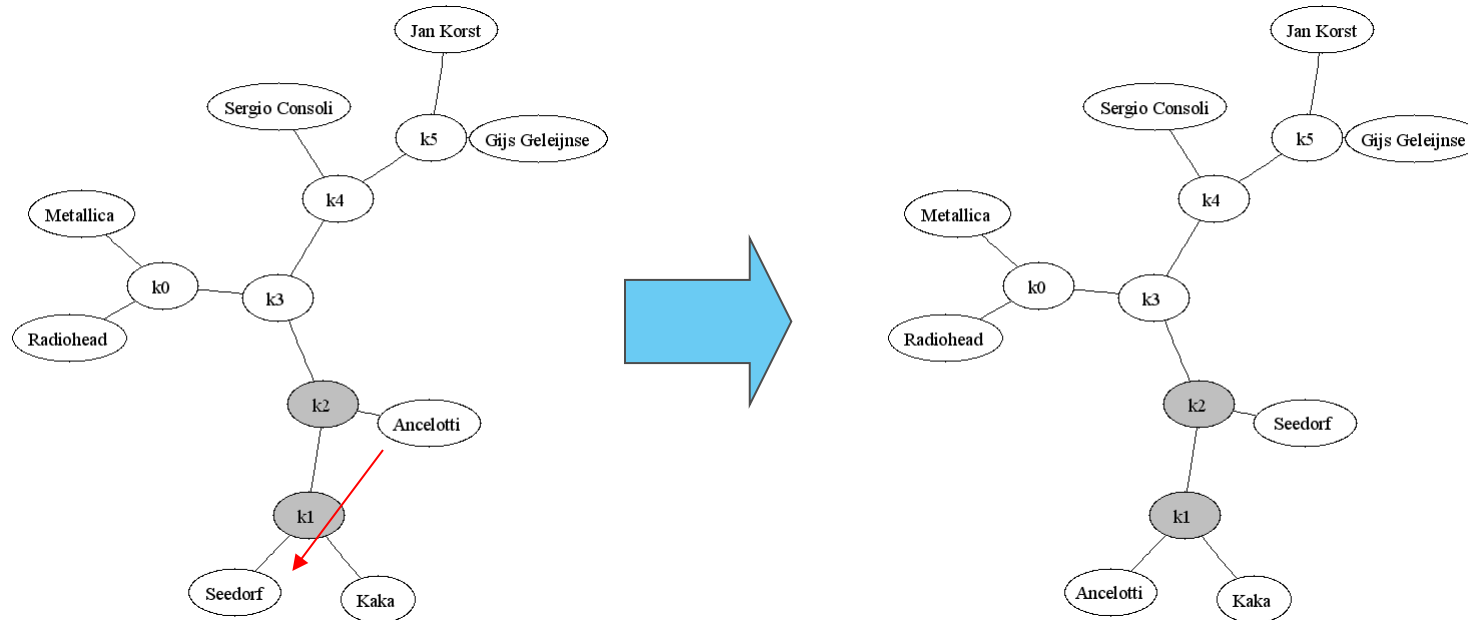
→ Exchange the leaf of the transition node with one of the terminal node



Base moves

One *terminal node* and one *transition node*

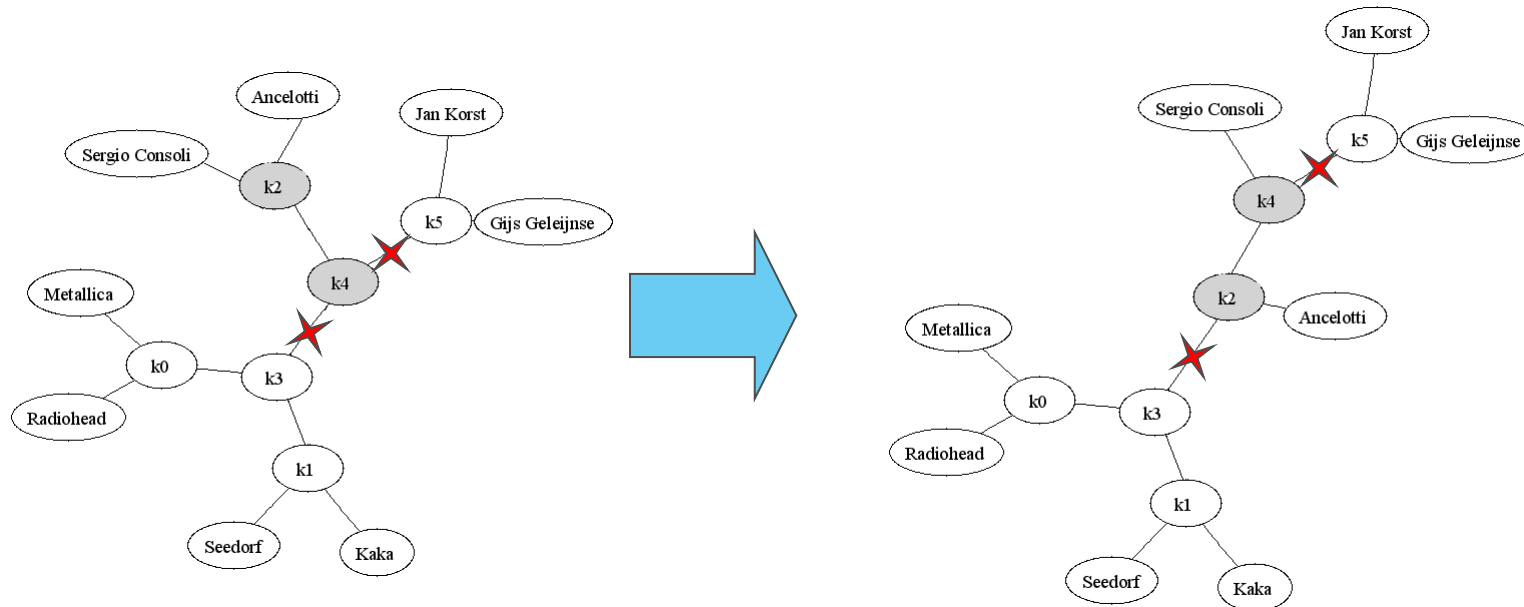
→ Exchange the leaf of the transition node with one of the terminal node



Base moves

One *terminal node* and one *cross node*

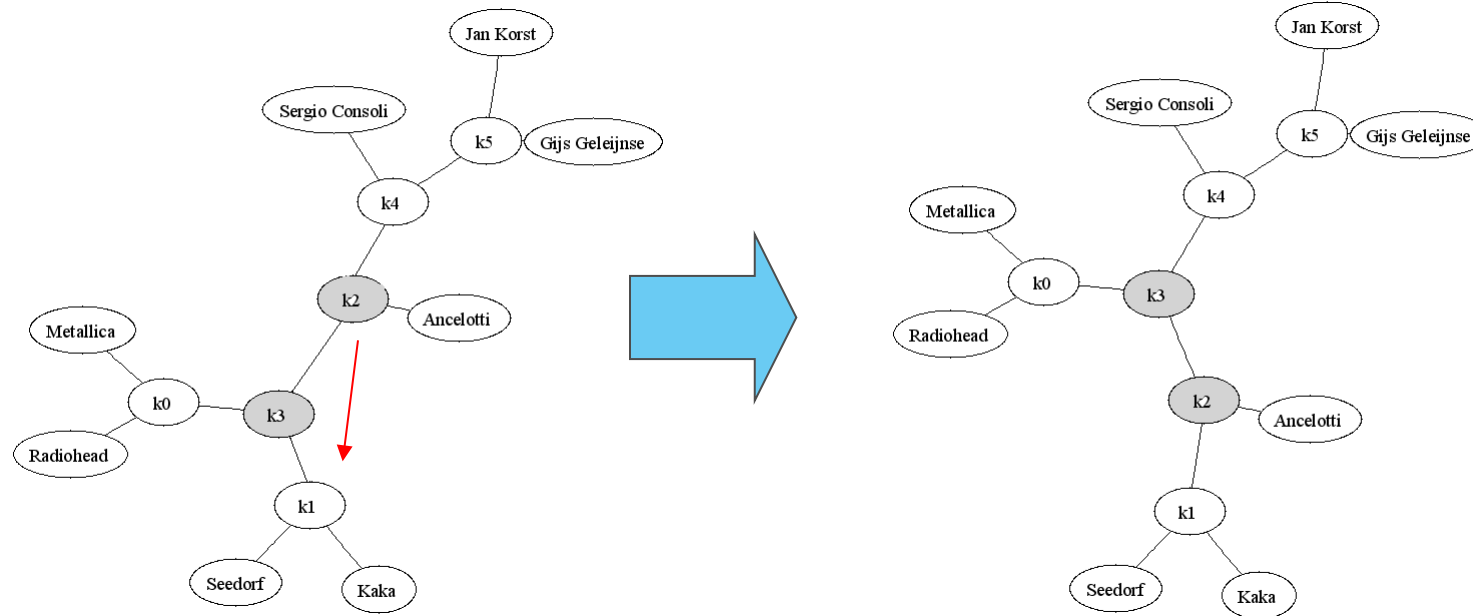
→ Transform into two *transition nodes*



Base moves

One *transition node* and one *cross node*

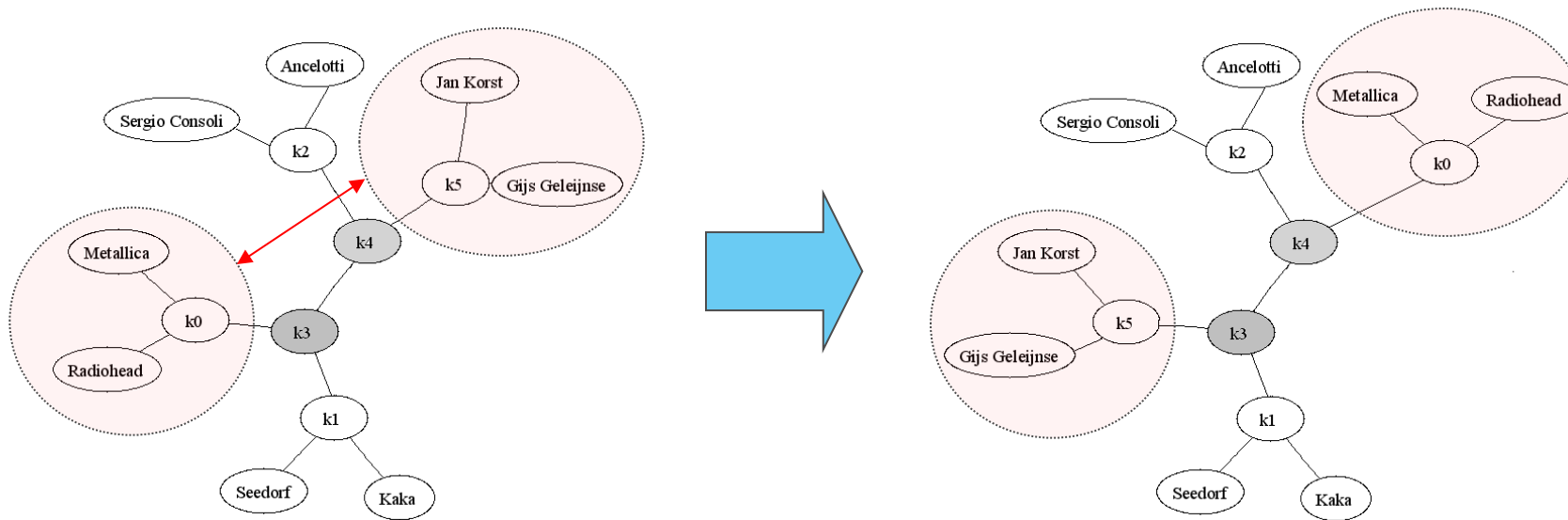
→ Move the *transition node* in one of the other two branches of the *cross node*



Base moves

Two consecutive *cross nodes*

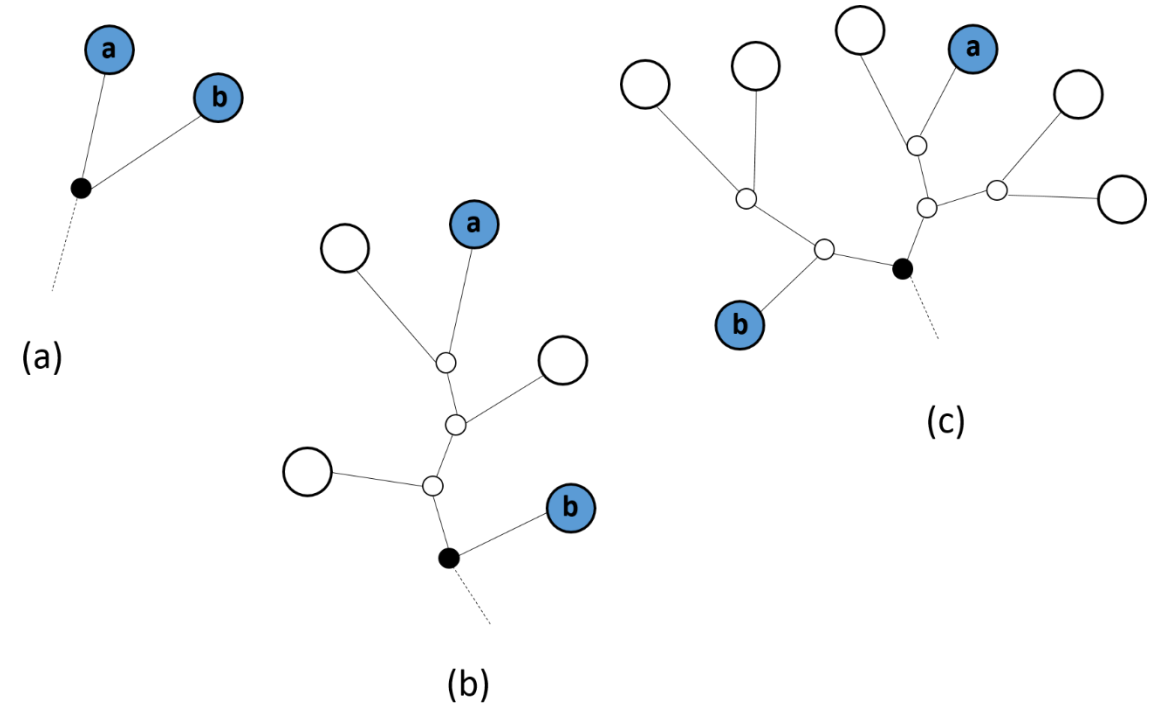
→ Try to exchange one of the subtree rooted at one cross node with another subtree rooted at the other cross node



Improved greedy constructive heuristic

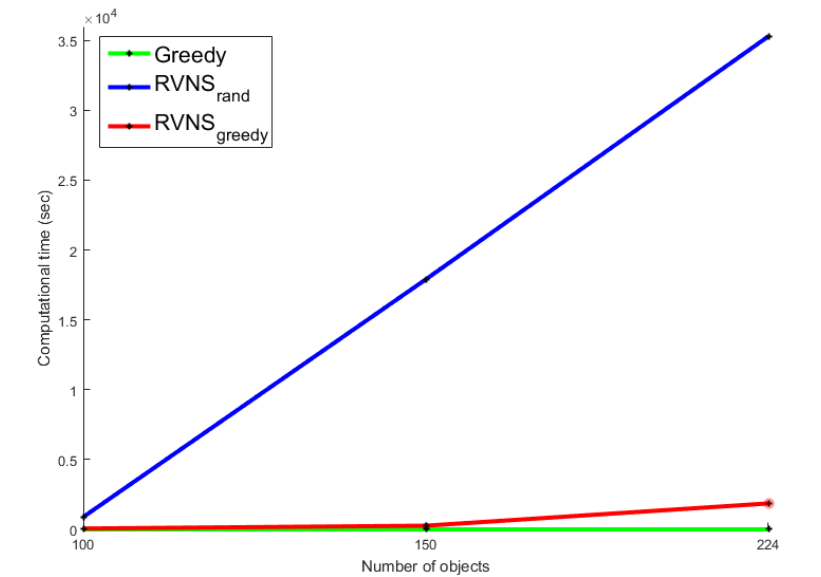
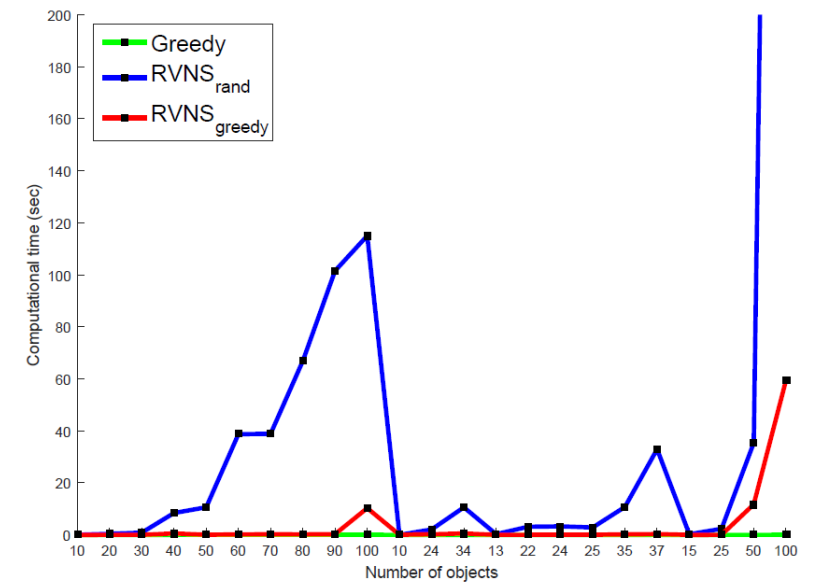
- MQTC metaheuristics → initial solution at random, or selecting flat structure
- Greedy constructive heuristic → initial starting solutions already of good quality and requiring short computational running time

- None of the two objects are already connected in t' , and therefore they are joined together by means of a terminal node (i.e. the dotted internal node in the figure);
- One of the two objects, in the example a , is already included as a leaf in t' , and therefore the other object b is linked to the subgraph in t' containing a by means of a transition node (i.e. the dotted internal node in the figure);
- Both objects are already included in t' but they belong to two different subgraphs, and therefore these two subgraphs containing respectively a and b are connected together by means of a cross node



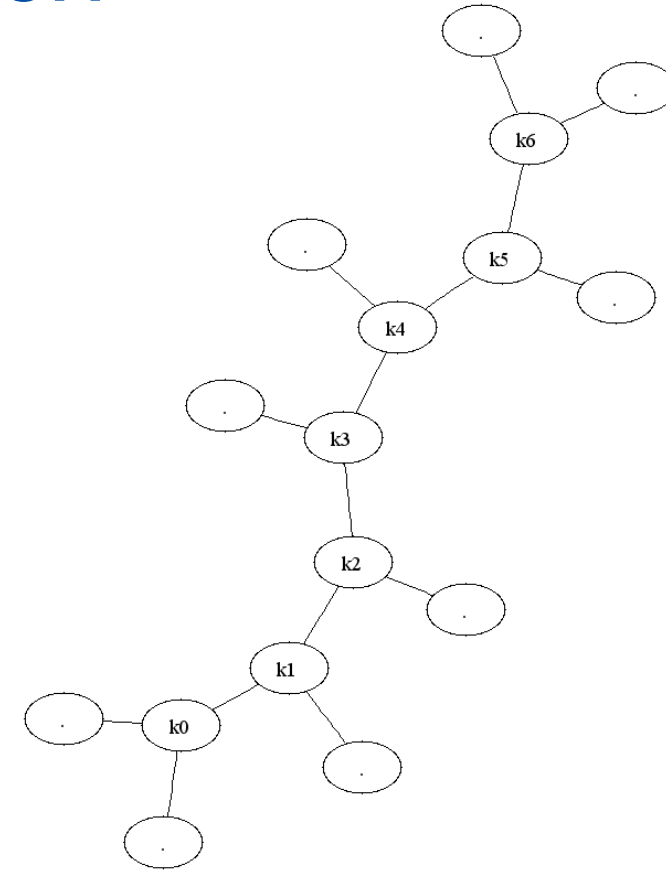
- It brings improvement of the overall performance of the MQTC heuristics deployed afterwards to the problem

<i>size n</i>	<u>Greedy</u>			<u>RVNS_{rand}</u>			<u>RVNS_{greedy}</u>		
	<i>cost</i>	<i>score</i>	<i>time</i>	<i>cost</i>	<i>score</i>	<i>time</i>	<i>cost</i>	<i>score</i>	<i>time</i>
<i>artificial</i>									
10	210.8000	0.89500	0.004	202.4000	1.00000	0.040	202.4000	1.00000	0.005
20	3301.2500	0.95270	0.003	3231.8500	1.00000	0.421	3231.8500	1.00000	0.006
30	15013.8656	0.92419	0.002	14559.4658	1.00000	0.861	14559.4658	1.00000	0.023
40	45245.7000	0.92400	0.006	43449.7500	1.00000	8.413	43449.7500	1.00000	0.554
50	104709.0800	0.85641	0.008	97207.8400	1.00000	10.606	97207.8400	1.00000	0.055
60	196608.3789	0.89166	0.012	186787.0474	1.00000	38.724	186787.0474	1.00000	0.176
70	360831.5064	0.88295	0.022	338182.8198	1.00000	38.858	338182.8198	1.00000	0.243
80	561067.4125	0.79739	0.028	509526.6875	1.00000	66.880	509526.6875	1.00000	0.193
90	806291.9392	0.89048	0.037	769344.2770	1.00000	101.512	769344.2770	1.00000	0.287
100	1232141.4400	0.92249	0.059	1178538.2000	1.00000	115.013	1178538.2000	1.00000	10.292
<i>nature</i>									
10	349.0720	0.99979	0.002	349.0720	0.99979	0.006	349.0720	0.99979	0.039
24	18649.3360	0.98524	0.002	18637.3390	0.99588	2.083	18637.3390	0.99588	0.232
34	82934.2444	0.98323	0.005	82922.0360	0.98792	10.610	82922.0360	0.98792	0.542
<i>geographical</i>									
13	476.4124	0.74265	0.002	439.7529	0.96843	0.270	439.7529	0.96843	0.016
22	4644.7111	0.82426	0.004	4377.1741	0.93507	3.140	4377.1741	0.93507	0.029
24	6839.0909	0.79911	0.004	6422.0182	0.92459	3.290	6422.0182	0.92459	0.044
25	8876.1814	0.79267	0.004	7827.1264	0.98760	2.840	7827.1264	0.98760	0.046
35	29209.7541	0.81774	0.004	26332.7485	0.98367	10.750	26332.7485	0.98367	0.205
37	28298.8724	0.79559	0.006	26846.2316	0.91973	32.940	26846.2316	0.91973	0.318
<i>musicians</i>									
15	2438.4025	0.93804	0.003	2434.1485	0.95273	0.250	2434.1485	0.95273	0.003
25	23785.9400	0.91853	0.005	23780.1875	0.92218	2.340	23780.1875	0.92218	0.007
50	445844.8230	0.90131	0.009	445504.8705	0.92252	35.370	445504.8705	0.92252	11.750
100	7700122.1360	0.86258	0.047	7696108.0255	0.88714	884.310	7695995.179	0.88783	59.367
150	40004637.3230	0.82152	0.209	39989276.7310	0.84614	17896.950	39988915.95	0.84672	262.368
224	202293353.0260	0.77282	0.930	202204482.6440	0.80849	35299.220	202203909.9695	0.80872	1857.483



Exact approach

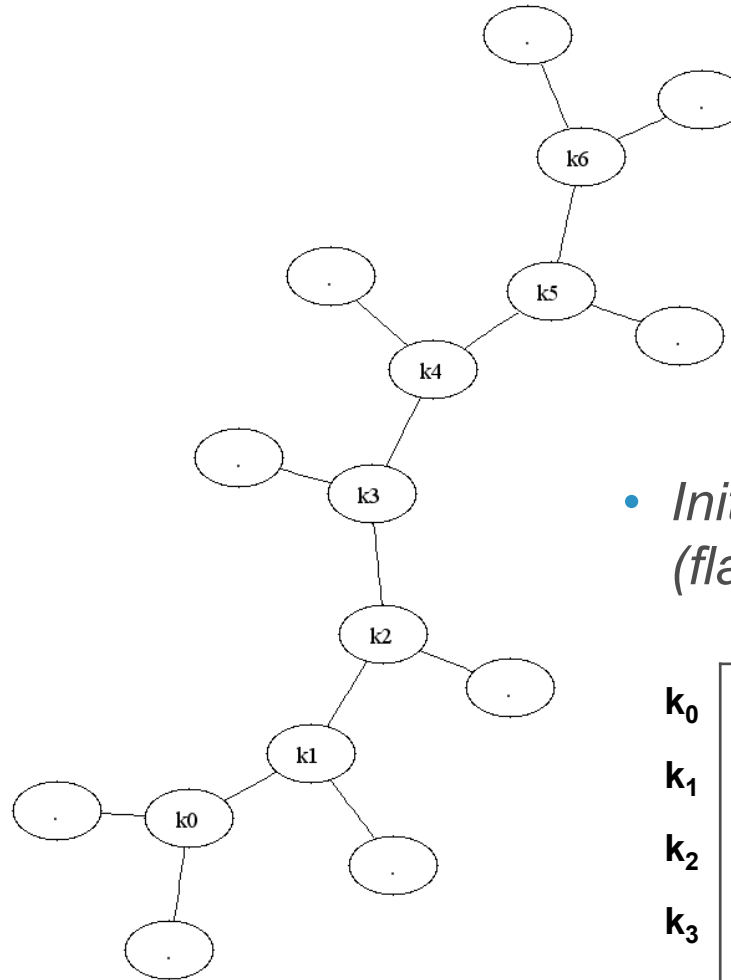
- **INPUT:** $distance_matrix(n,n)$
- $t(structure) \leftarrow INITIALIZATION(n);$
- $best_t \leftarrow t;$
- **While** ($t(structure) \neq NULL$)
 - $t \leftarrow ASSIGN_LEAVES(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If**($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$**Repeat**
 - $t \leftarrow NEXT_PERMUTATION(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If**($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$
 - **Until**($NO_FURTHER_PERMUTATIONS(t,leaves[n])$)
 - $t(structure) \leftarrow NEXT_STRUCTURE(t);$
- **OUTPUT:** $best_t$



Data representation

- Complete matrix (e.g. with $n = 9$ leaves):

	k_0	k_1	k_2	k_3	k_4	k_5	k_6	7	8	9	10	11	12	13	14	15
k_0	STRUCTURE SUBMATRIX															
k_1																
k_2																
k_3																
k_4																
k_5																
k_6																
7	LEAVES SUBMATRIX															
8																
9																
10																
11																
12																
13																
14																
15																
	COEFFICIENTS SUBMATRIX															

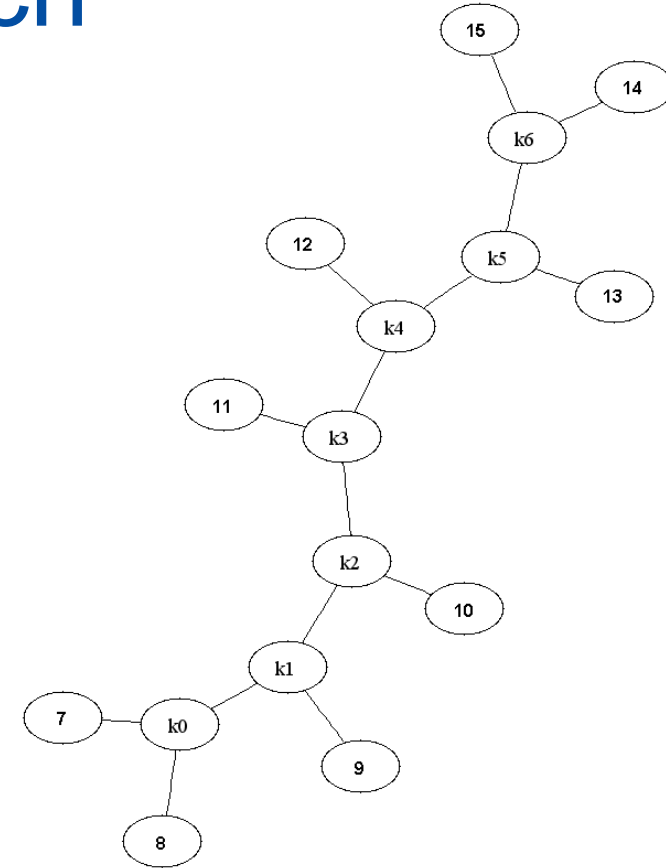


- Initial Structure Submatrix (flat structure)

	k_0	k_1	k_2	k_3	k_4	k_5	k_6
k_0	1	1	0	0	0	0	0
k_1	1	2	1	0	0	0	0
k_2	0	1	2	1	0	0	0
k_3	0	0	1	2	1	0	0
k_4	0	0	0	1	2	1	0
k_5	0	0	0	0	1	2	1
k_6	0	0	0	0	0	1	1

Exact approach

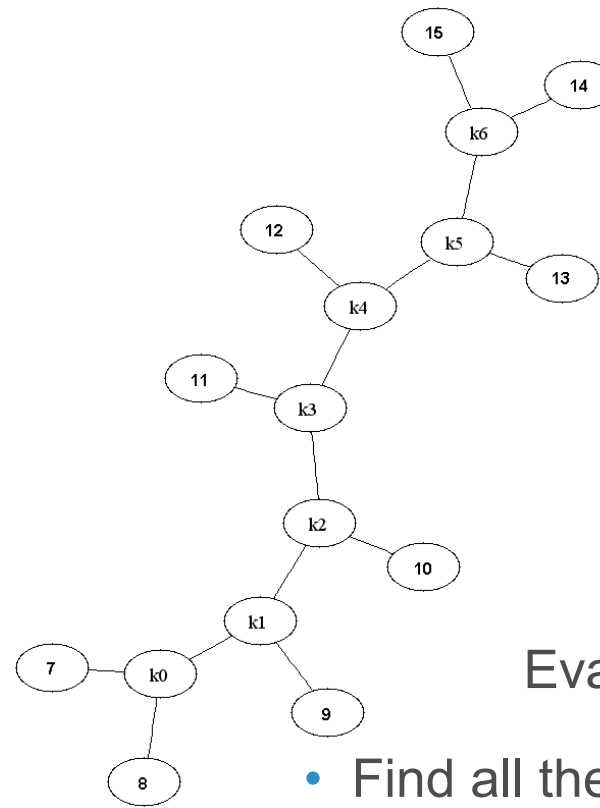
- **INPUT:** $distance_matrix(n,n)$
- $t(structure) \leftarrow INITIALIZATION(n);$
- $best_t \leftarrow t;$
- **While** ($t(structure) \neq NULL$)
 - $t \leftarrow ASSIGN_LEAVES(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If**($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$**Repeat**
 - $t \leftarrow NEXT_PERMUTATION(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If**($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$
 - **Until**($NO_FURTHER_PERMUTATIONS(t,leaves[n])$)
 - $t(structure) \leftarrow NEXT_STRUCTURE(t);$
- **OUTPUT:** $best_t$



Assign leaves

- Modify the *leaves submatrix*:

	k ₀	k ₁	k ₂	k ₃	k ₄	k ₅	k ₆	7	8	9	10	11	12	13	14	15
k ₀	1	1	0	0	0	0	0	1	1	0	0	0	0	0	0	0
k ₁	1	2	1	0	0	0	0	0	0	1	0	0	0	0	0	0
k ₂	0	1	2	1	0	0	0	0	0	0	1	0	0	0	0	0
k ₃	0	0	1	2	1	0	0	0	0	0	0	1	0	0	0	0
k ₄	0	0	0	1	2	1	0	0	0	0	0	0	1	0	0	0
k ₅	0	0	0	0	1	2	1	0	0	0	0	0	0	1	0	0
k ₆	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1
7	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
8	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
9	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0
10	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0
11	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0
12	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0
13	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0
14	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0
15	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1



Evaluate the cost function:

- Find all the $\binom{n}{4}$ quartets embedded in t
- Consider the $n \times n$ symmetric distance matrix, sum the costs $d_{i,j}$ of each embedded quartet:

$$= 21 \cdot d(7,8) + 15 \cdot d(7,9) + 10 \cdot d(7,8) + \dots$$

$$\dots + 10 \cdot d(12,15) + 15 \cdot d(13,15) + 21 \cdot d(14,8);$$

Evaluate the cost function

$$C_{TOT} = [d(7,8) + d(9,10)] + \dots + [d(12,13) + d(14,15)] =$$

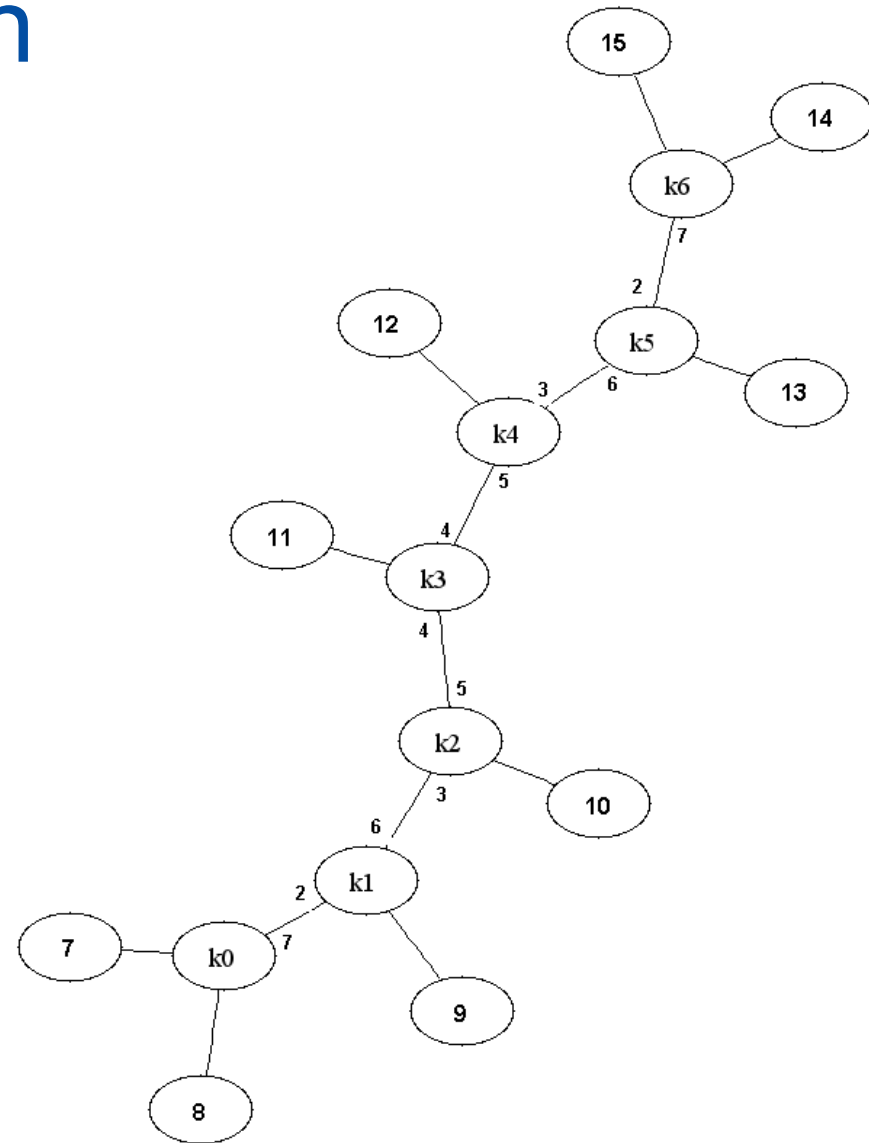
$$= 21 \cdot d(7,8) + 15 \cdot d(7,9) + 10 \cdot d(7,8) + \dots$$

$$\dots + 10 \cdot d(12,15) + 15 \cdot d(13,15) + 21 \cdot d(14,8);$$

→ Coefficients evaluated by means of an efficient procedure (polynomial)

- Consider the path between two leaves i and j
- Consider the number of leaves outgoing the edges k which are not contained in the subgraph between i and j
- For each k , call this number: $\alpha(i,j,k)$
- Each coefficient associated to the distance $d(i,j)$ is the sum of the triangular numbers T of each $\alpha(i,j,k)$, $\forall k$, i.e.:

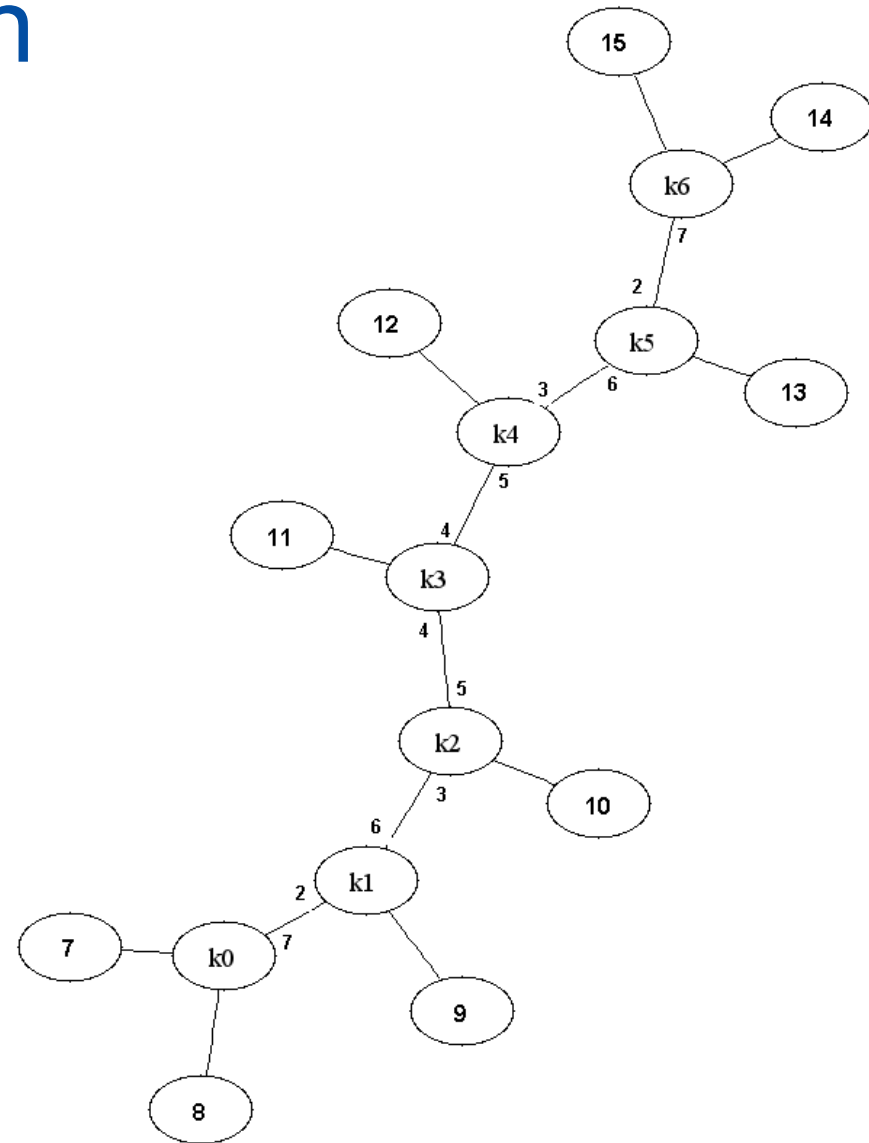
$$\text{coeff}(i,j) = \sum_k T(\alpha(i,j,k)) = \sum_k \frac{\alpha(i,j,k) (\alpha(i,j,k) + 1)}{2}$$



Evaluate the cost function

- Evaluate the coefficients using *backtracking*
- Insert them in the *coefficients submatrix*:

	k ₀	k ₁	k ₂	k ₃	k ₄	k ₅	k ₆	7	8	9	10	11	12	13	14	15
k ₀	1	7	0	0	0	0	0	1	1	0	0	0	0	0	0	0
k ₁	2	2	6	0	0	0	0	0	0	1	0	0	0	0	0	0
k ₂	0	3	2	5	0	0	0	0	0	0	1	0	0	0	0	0
k ₃	0	0	4	2	4	0	0	0	0	0	0	1	0	0	0	0
k ₄	0	0	0	5	2	3	0	0	0	0	0	0	1	0	0	0
k ₅	0	0	0	0	6	2	2	0	0	0	0	0	0	1	0	0
k ₆	0	0	0	0	0	7	1	0	0	0	0	0	0	0	1	1
7	1	0	0	0	0	0	0	1	21	15	10	6	3	1	0	0
8	1	0	0	0	0	0	0	21	1	15	10	6	3	1	0	0
9	0	1	0	0	0	0	0	15	15	1	11	7	4	2	1	1
10	0	0	1	0	0	0	0	10	10	11	1	9	6	4	3	3
11	0	0	0	1	0	0	0	6	6	7	9	1	9	7	6	6
12	0	0	0	0	1	0	0	3	3	4	6	9	1	11	10	10
13	0	0	0	0	0	1	0	1	1	2	4	7	11	1	15	15
14	0	0	0	0	0	0	1	0	0	1	3	6	10	15	1	21
15	0	0	0	0	0	0	1	0	0	1	3	6	10	15	0	1



Triangular numbers

- The triangular number of n , referred to as $T(n)$, where n is any natural number, is the sum of the n natural numbers from 1 to n , i.e.:

$$T(n) = 1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + n = \frac{n(n+1)}{2}$$

- $T(1) = 1$
- $T(2) = 1 + 2 = 3$
- $T(3) = 1 + 2 + 3 = 6$
- $T(4) = 1 + 2 + 3 + 4 = 10$
-

- The properties of such numbers were first studied by ancient Greek mathematicians, particularly the Pythagoreans. Flocks of birds often fly in this triangular formation. Even several airplanes when flying together constitute this fascinating swarm sequence (Deza et al., 2012).
- We found this particular relation for the coefficients stored in Coefficients sub-matrix

Exact approach

- **INPUT:** $distance_matrix(n,n)$
 - $t(structure) \leftarrow INITIALIZATION(n);$
 - $best_t \leftarrow t;$
 - **While** ($t(structure) \neq NULL$)
 - $t \leftarrow ASSIGN_LEAVES(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If** ($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$
 - **Repeat**
 - $t \leftarrow NEXT_PERMUTATION(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If** ($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$
 - **Until** ($NO_FURTHER_PERMUTATIONS(t,leaves[n])$)
 - $t(structure) \leftarrow NEXT_STRUCTURE(t);$
 - **OUTPUT:** $best_t$

Diagram illustrating a graph structure and a matrix representation. The matrix is divided into two main sections by a vertical dashed line. The left section has columns k_0 to k_6 and rows k_0 to k_6 . The right section has columns 7 to 15 and rows 7 to 15. The matrix contains numerical values, some in red. A graph structure is overlaid on the matrix, with nodes represented by circles containing numbers and edges represented by lines. Some nodes are labeled k_0 through k_6 . A red arrow points from the bottom left towards the bottom right, passing through a node labeled 9.

	k_0	k_1	k_2	k_3	k_4	k_5	k_6	7	8	9	10	11	12	13	14	15
k_0	1	7	0	0	0	0	0	1	1	0	0	0	0	0	0	0
k_1	2	2	6	0	0	0	0	0	0	1	0	0	0	0	0	0
k_2	0	3	2	5	0	0	0	0	0	0	1	0	0	0	0	0
k_3	0	0	4	2	4	0	0	0	0	0	0	0	0	0	0	0
k_4	0	0	0	5	2	3	0	0	0	0	0	0	1	0	0	0
k_5	0	0	0	0	6	2	0	0	0	0	0	0	0	1	0	0
k_6	0	0	0	0	0	7	0	0	0	0	0	0	0	0	1	1
7	1	0	0	0	0	0	0	1	21	15	10	6	3	1	0	0
8	1	0	0	0	0	0	0	21	1	15	10	6	3	1	0	0
9	0	1	0	0	0	0	0	15	15	1	11	7	4	2	1	1
10	0	0	1	0	0	0	0	10	10	11	1	10	9	6	4	3
11	0	0	0	1	0	0	0	6	6	7	9	1	9	7	6	6
12	0	0	0	0	1	0	0	3	3	4	6	9	1	11	10	10
13	0	0	0	0	0	0	0	1	1	2	4	7	11	1	15	15
14	0	0	0	0	0	0	1	0	0	1	3	6	10	15	1	21
15	0	0	0	0	0	0	1	0	0	↑	3	6	10	15	0	1

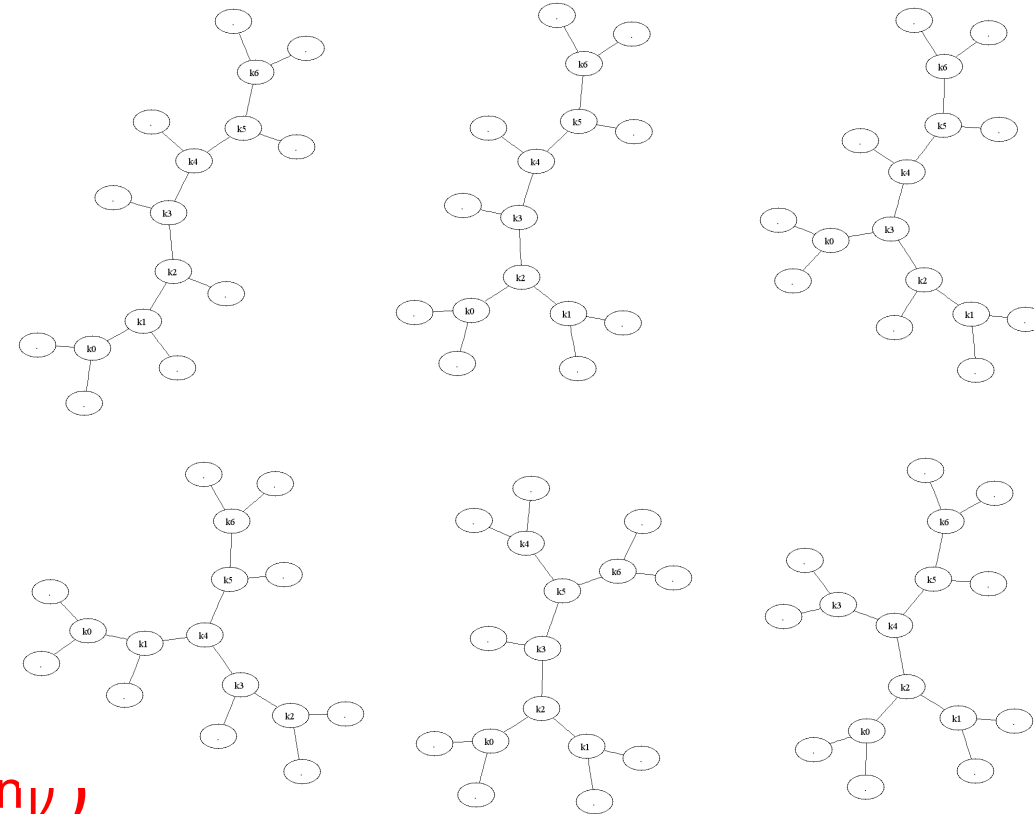
Exact approach

- **INPUT:** $distance_matrix(n,n)$
- $t(structure) \leftarrow INITIALIZATION(n);$
- $best_t \leftarrow t;$
- **While** ($t(structure) \neq NULL$)
 - $t \leftarrow ASSIGN_LEAVES(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If** ($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$

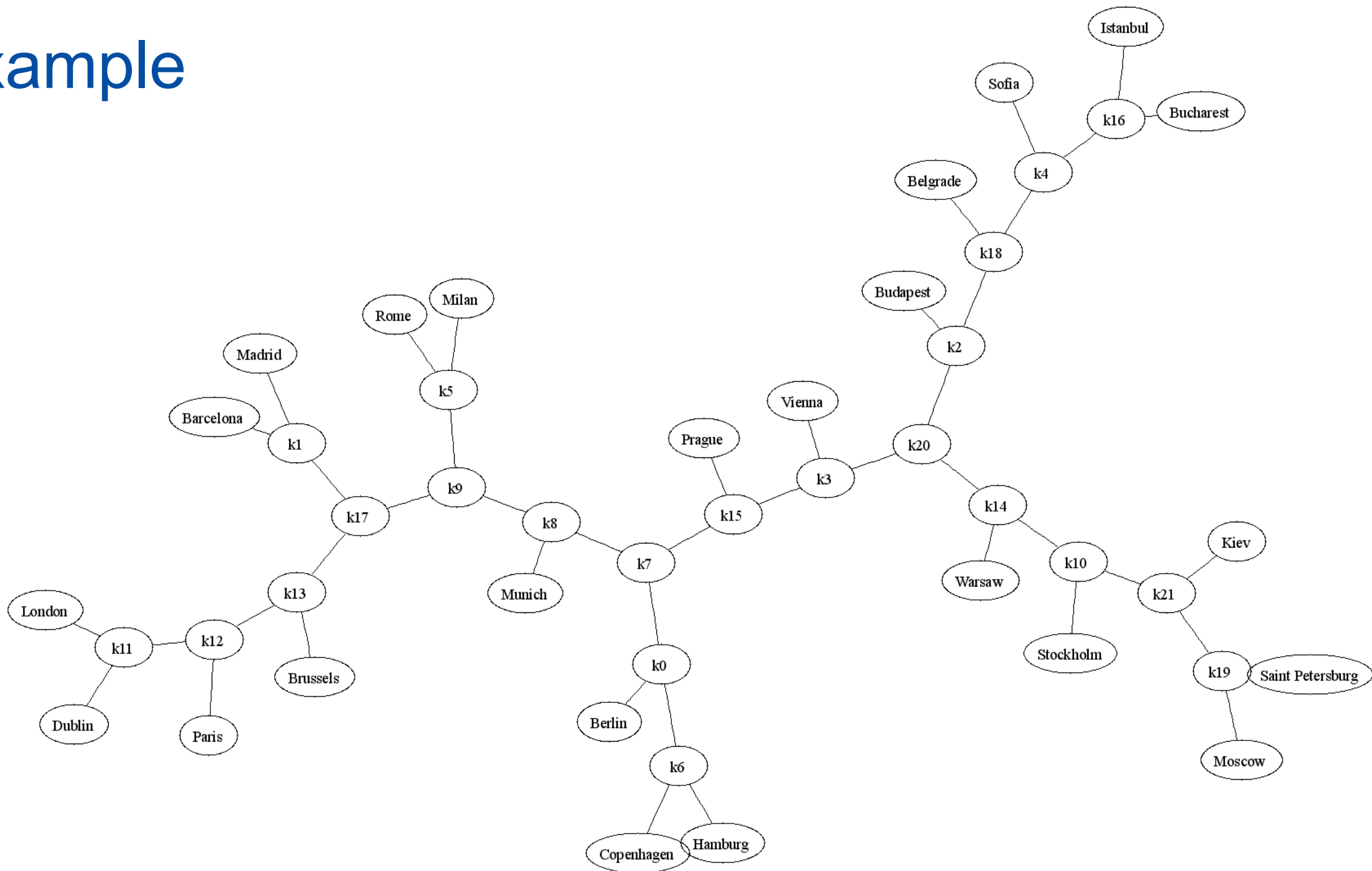
Repeat

- $t \leftarrow NEXT_PERMUTATION(leaves[n]);$
 - $C(t) \leftarrow EVALUATE_COST_FUNCTION(t);$
 - **If** ($C(t) < C(best_t)$) 1) $best_t \leftarrow t;$
- **Until** ($NO_FURTHER_PERMUTATIONS(t,leaves[n])$,
- $t(structure) \leftarrow NEXT_STRUCTURE(t);$
- **OUTPUT:** $best_t$

E.g. of the 6 different boron tree structures with 9 leaves



Example



Generated first exact benchmark for the MQTC problem

Cost function values $C(t)$

n	A000672	$n!$	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
4	1	24	1.2562	1.4909	1.4576	1.8957	1.4154	1.2452	1.6934	1.9603	1.0432	1.7008
5	1	120	6.5122	5.8076	5.6784	6.1737	6.3538	5.8861	6.5846	6.6457	6.0696	7.3811
6	2	720	24.0303	19.2474	23.2273	21.0043	21.0214	19.0146	17.2770	17.9369	21.2395	19.1279
7	2	5040	44.9921	44.6811	42.3670	46.4926	50.2484	43.1167	48.8859	50.2999	46.3158	48.8200
8	4	$40 \cdot 10^3$	91.9606	109.9054	86.5335	99.3844	105.2160	97.3845	77.2696	103.9752	83.4889	109.4040
9	6	$362 \cdot 10^3$	190.8828	154.3390	163.5998	222.8461	196.3194	182.3488	197.6758	171.6653	215.1805	188.2427
10	11	$3.6 \cdot 10^6$	284.0047	309.9597	342.0803	320.2249	262.7439	280.4022	342.7556	343.7522	272.3572	319.8787
11	18	$39 \cdot 10^6$	447.3903	489.2708	433.1529	426.7213	527.8555	485.8953	535.1013	489.3120	506.9666	505.0629
12	37	$479 \cdot 10^6$	791.3053	581.4883	670.5000	753.8792	723.5195	818.7593	742.6702	768.6850	743.3466	728.3262

Computational running times (s)

n	A000672	$n!$	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
4	1	24	0.005	0.002	0.003	0.003	0.004	0.002	0.005	0.002	0.002	0.002
5	1	120	0.015	0.017	0.014	0.010	0.017	0.018	0.015	0.029	0.022	0.011
6	2	720	0.014	0.118	0.018	0.057	0.104	0.046	0.043	0.052	0.013	0.067

Computational running times (s)

n	A000672	$n!$	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
7	2	5040	0.326	0.348	0.352	0.302	0.324	0.320	0.304	0.279	0.269	0.282
8	4	$40 \cdot 10^3$	3.599	3.196	3.212	3.224	3.253	3.272	3.437	3.247	3.310	3.247
9	6	$362 \cdot 10^3$	45.143	45.644	48.253	41.139	43.678	48.791	44.454	45.321	54.439	46.408
10	11	$3.6 \cdot 10^6$	231.618	240.738	230.576	230.057	230.569	230.084	230.228	230.233	232.964	230.493
11	18	$39 \cdot 10^6$	$4.179 \cdot 10^3$	$4.134 \cdot 10^3$	$4.133 \cdot 10^3$	$4.128 \cdot 10^3$	$4.126 \cdot 10^3$	$4.124 \cdot 10^3$	$4.121 \cdot 10^3$	$4.127 \cdot 10^3$	$4.190 \cdot 10^3$	$4.134 \cdot 10^3$
12	37	$479 \cdot 10^6$	$102.442 \cdot 10^3$	$102.916 \cdot 10^3$	$101.953 \cdot 10^3$	$101.885 \cdot 10^3$	$98.877 \cdot 10^3$	$105.692 \cdot 10^3$	$96.052 \cdot 10^3$	$101.512 \cdot 10^3$	$100.534 \cdot 10^3$	$104.951 \cdot 10^3$

Conclusions

- We have introduced the MQTC problem, the mathematical paradigm at the base of the quartet method of hierarchical clustering
- Discussed the best metaheuristics to date
- Presented details of an exact solution approach
 - Exact solutions for small problem instances (first existing benchmark)
- The construction mechanisms of the MQTC algorithms seem to be rather suitable for a quantum algorithmic implementation
 - Very interesting concepts and insights which may be useful for the construction of efficient quantum computing methods for the problem, obtained either by improving the MQTC metaheuristics to date, or by hybridization with the exact solution approach

References

- Cilibrasi R, Vitányi PMB (2005) Clustering by compression. IEEE Transactions on Information Theory 51(4):1523–1545
- Cilibrasi R, Vitányi PMB (2007) The Google similarity distance. IEEE Transactions on Knowledge and Data Engineering 19(3):370–383
- Costa, L. R., Aloise, D., Mladenović, N., 2017. Less is more: basic variable neighborhood search heuristic for balanced minimum sum-of-squares clustering. Information Sciences 415 - 416, 247 – 253.
- S. Consoli, J. Korst, S. Pauws, and G. Geleijnse, 2019. An exact algorithm for the minimum quartet tree cost problem. 4OR: A Quarterly Journal of Operations Research, 17(4), 401-425
- S. Consoli and N. I. Stilianakis, 2017. A quartet method based on variable neighbourhood search for biomedical literature extraction and clustering. International Transactions in Operational Research, 24(3), 537-558
- Consoli S, Darby-Dowman K, Geleijnse G, Korst J, Pauws S (2010) Heuristic approaches for the quartet method of hierarchical clustering. IEEE Transactions on Knowledge and Data Engineering 22(10):1428– 1443
- S. Consoli, J. Korst, S. Pauws, and G. Geleijnse, 2020. Improved metaheuristics for the quartet method of hierarchical clustering. Journal of Global Optimization, 78(2), 241-270
- Deza E., Deza M. M. (2012) Figurate numbers. World Scientific Publishing, Singapore.
- Sloane N. J. A., Plouffe S. (1995) The Encyclopedia of Integer Sequences. Academic Press, San Diego, CA.

Thank you

Any questions?

You can find me ***sergio.consoli@ec.europa.eu***



© European Union 2020

Unless otherwise noted the reuse of this presentation is authorised under the [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/) license. For any use or reproduction of elements that are not owned by the EU, permission may need to be sought directly from the respective right holders.